

第一部分：理论知识

1.1 为什么需要学习磁盘管理？

生活化类比：房屋装修

想象你刚买了一套毛坯房（新硬盘），需要装修才能入住：

装修步骤	磁盘管理对应操作
规划房间布局	磁盘分区（Partitioning）
铺地板/贴瓷砖	创建文件系统（Formatting）
开门通道	挂载（Mounting）
搬家具入住	存储数据
扩建/改造	扩容/LVM管理

核心概念：

- 分区：将一块硬盘划分为多个独立区域
- 文件系统：规定数据如何在分区中存储和组织
- 挂载：将分区连接到目录树，使其可被访问

1.2 Linux 磁盘设备命名规则

配图 1：磁盘设备命名规则

Linux 磁盘设备命名规则

设备文件位置

/dev/ - 所有设备文件所在目录
/dev/sda - 第一块 SATA/SAS/SCSI 硬盘
/dev/sda1 - 第一块硬盘的第一个分区
/dev/nvme0n1 - 第一块 NVMe 固态硬盘
/dev/nvme0n1p1 - NVMe 硬盘的第一个分区

SATA / SAS / SCSI 磁盘命名

/dev/sda1

sd
SCSI Disk
磁盘类型

a
第几块盘
a=1,b=2...

1
分区号
1,2,3...

示例

/dev/sda → 第1块硬盘
/dev/sdb → 第2块硬盘
/dev/sdc → 第3块硬盘
/dev/sda1 → 第1块盘第1分区
/dev/sda2 → 第1块盘第2分区
/dev/sdb1 → 第2块盘第1分区

NVMe 固态硬盘命名

/dev/nvme0n1p1

nvme0
控制器编号
0,1,2...

n1
命名空间
(磁盘号)

p1
分区号
p1,p2...

示例

/dev/nvme0n1 → 第1个NVMe盘
/dev/nvme1n1 → 第2个NVMe盘
/dev/nvme0n1p1 → 第1盘第1分区
/dev/nvme0n1p2 → 第1盘第2分区
/dev/nvme1n1p1 → 第2盘第1分区

虚拟机/云服务器磁盘

VirtIO 磁盘 (KVM虚拟机):
/dev/vda, /dev/vdb, /dev/vda1...

Xen 虚拟磁盘:
/dev/xvda, /dev/xvdb, /dev/xvda1...

云服务器常见:
阿里云/腾讯云: /dev/vda (系统盘)
AWS: /dev/xvda 或 /dev/nvme*

查看磁盘设备的命令

```
# 列出所有块设备
lsblk

# 查看磁盘分区表
fdisk -l

# 查看磁盘使用情况
df -h

# 查看设备文件
ls -l /dev/sd* /dev/nvme*
```

分区编号规则 (MBR 分区表)

主分区 (Primary): 1-4
扩展分区 (Extended): 占用 1-4 中的一个
逻辑分区 (Logical): 从 5 开始

示例:
/dev/sda1 → 主分区1
/dev/sda2 → 主分区2
/dev/sda3 → 扩展分区
/dev/sda5 → 逻辑分区1 (注意跳过了4)
/dev/sda6 → 逻辑分区2

380 x 151

图例:

SATA/SAS

NVMe

虚拟磁盘

命令

分区规则

命名规则总结

磁盘类型	设备名格式	示例
SATA/SAS/SCSI	/dev/sd[a-z]	/dev/sda, /dev/sdb
NVMe SSD	/dev/nvme[0-9]n[1-9]	/dev/nvme0n1, /dev/nvme1n1
VirtIO (虚拟机)	/dev/vd[a-z]	/dev/vda, /dev/vdb
Xen 虚拟磁盘	/dev/xvd[a-z]	/dev/xvda, /dev/xvdb
IDE (老式)	/dev/hd[a-z]	/dev/hda, /dev/hdb

1.3 分区表类型：MBR vs GPT

配图 2：MBR 与 GPT 对比

MBR 与 GPT 分区表对比

MBR (Master Boot Record)

主引导记录 - 传统分区方案

MBR 512B

主分区 1 sda1

主分区 2 sda2

主分区 3 sda3

扩展分区 (sda4)

逻辑5

逻辑6

逻辑7

✓ 特点

- 兼容性好，支持老旧系统
- 引导记录仅 512 字节
- 最多 4 个主分区
- 使用扩展分区可创建更多逻辑分区
- 兼容 BIOS 和 UEFI 引导
- 工具: fdisk

✗ 限制

- 最大支持 2TB 磁盘
- 最多 4 个主分区
- 分区表无备份，损坏难恢复
- 不支持 UEFI 安全启动
- 32 位寻址限制

GPT (GUID Partition Table)

全局唯一标识分区表 - 现代分区方案

保护 MBR

GPT 头

分区表

分区 1 sda1

分区 2 sda2

分区 3 sda3

...更多分区...

备份表

备份头

备份GPT头

✓ 优势

- 支持超大磁盘 (18EB)
- 最多 128 个分区 (无主/扩展之分)
- 使用 GUID 唯一标识分区
- 分区表有备份，更安全
- 支持 UEFI 安全启动
- CRC32 校验，数据完整性好
- 工具: gdisk, parted

🔥 适用场景

- 磁盘容量超过 2TB
- 需要超过 4 个分区
- UEFI 启动的系统
- 现代服务器和工作站
- 企业级存储系统
- 所有新部署的系统 (推荐)

MBR vs GPT 对比表

对比项	MBR	GPT	推荐使用
最大磁盘容量	2 TB	18 EB (约 18,000 PB)	>2TB 必须用 GPT
最大分区数	4 主分区 (或 3主+1扩展)	128 个分区	需要多分区用 GPT
启动方式	BIOS / UEFI (兼容模式)	UEFI (原生支持)	UEFI 系统用 GPT
分区工具	fdisk	gdisk, parted	parted 两者都支持

🔑 结论：新系统推荐使用 GPT，除非需要兼容老旧 BIOS 系统

快速对比表

特性	MBR	GPT
最大磁盘容量	2 TB	18 EB
最大分区数	4 个主分区	128 个分区
分区表备份	无	有（磁盘末尾）
启动方式	BIOS/UEFI	UEFI
分区工具	fdisk	gdisk, parted
适用场景	老旧系统、小磁盘	现代系统、大磁盘

1.4 常见文件系统类型

配图 3：Linux 文件系统对比

Linux 常见文件系统对比



EXT4

Fourth Extended Filesystem

- ✓ Linux 默认文件系统
- ✓ 成熟稳定，兼容性好
- ✓ 最大卷 1EB，文件 16TB
- ✓ 支持日志，崩溃恢复快
- ✓ 向后兼容 ext2/ext3

适用场景：

- 通用服务器系统盘
- 普通数据存储
- 虚拟机磁盘

创建命令：

```
mkfs.ext4 /dev/sdb1
```



XFS

High-Performance Filesystem

- ✓ RHEL 7+ 默认文件系统
- ✓ 高性能，大文件优化
- ✓ 最大卷 8EB，文件 8EB
- ✓ 支持在线扩容（不能缩小）
- ✓ 优秀的并行 I/O

适用场景：

- 大文件存储（视频/备份）
- 高性能数据库
- 大数据处理

创建命令：

```
mkfs.xfs /dev/sdb1
```



Btrfs

B-Tree Filesystem (Butter FS)

- ✓ 下一代文件系统
- ✓ 内置快照和克隆功能
- ✓ 透明压缩（节省空间）
- ✓ 支持 RAID 和子卷
- ✓ 数据校验和自修复

适用场景：

- 需要快照备份
- 容器存储
- 开发测试环境

创建命令：

```
mkfs.btrfs /dev/sdb1
```



Swap

交换空间 (虚拟内存)

- ✓ 虚拟内存扩展
- ✓ 内存不足时使用
- ✓ 支持休眠功能

推荐大小：

- 内存 ≤2G: 2倍内存
- 内存 2-8G: 等于内存
- 内存 >8G: 8G 或更少

创建命令：

```
mkswap /dev/sdb2  
swapon /dev/sdb2
```

文件系统特性对比

特性	EXT4	XFS	Btrfs	NTFS	FAT32/exFAT
最大卷大小	1 EB	8 EB	16 EB	16 EB	2TB / 128PB
最大文件	16 TB	8 EB	16 EB	16 TB	4GB / 16EB
在线扩容	✓ 支持	✓ 支持 (只能扩大)	✓ 支持 (扩/缩)	✗ 不支持	✗ 不支持
快照支持	✗ 需要 LVM	✗ 需要 LVM	✓ 内置支持	✗ 不支持	✗ 不支持
默认系统	Ubuntu/Debian	RHEL 7+ / Rocky	openSUSE/Fedora	Windows	U盘/移动设备
Linux 支持	✓ 完整支持	✓ 完整支持	✓ 完整支持	⚠ ntfs-3g	✓ 完整支持

选择建议

- ✗ RHEL/CentOS/Rocky: 推荐使用 XFS
- ✗ 通用 Linux: EXT4 足够使用
- ✗ 需要快照: 使用 Btrfs 或 LVM
- ✗ 与 Windows 共享: 使用 exFAT

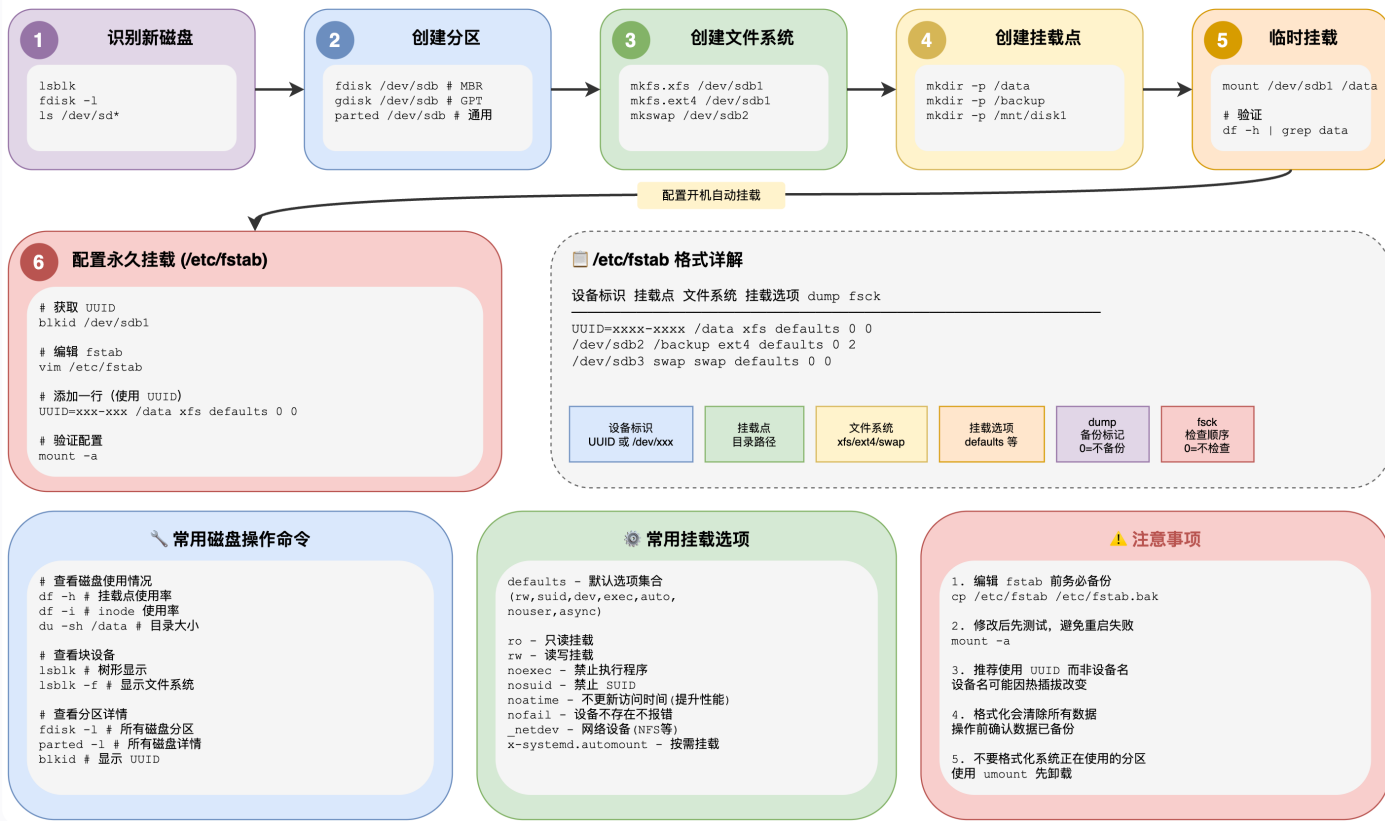
文件系统选择指南

使用场景	推荐文件系统	理由
RHEL/CentOS/Rocky 系统盘	XFS	系统默认，性能优秀
通用数据存储	EXT4 或 XFS	成熟稳定
大文件存储（视频/备份）	XFS	大文件性能优化
需要快照备份	Btrfs 或 LVM+EXT4	内置快照功能
数据库服务器	XFS	高并发 I/O
与 Windows 共享	exFAT	跨平台兼容
U 盘/移动设备	exFAT	无文件大小限制

1.5 磁盘管理流程总览

配图 4：磁盘管理完整流程

Linux 磁盘管理完整流程



流程总结

新磁盘 → 分区 → 创建文件系统 → 创建挂载点 → 挂载 → 配置 fstab

步骤	命令示例	说明
1. 识别磁盘	<code>lsblk</code>	确认新磁盘设备名
2. 创建分区	<code>fdisk /dev/sdb</code>	MBR 分区表
3. 创建文件系统	<code>mkfs.xfs /dev/sdb1</code>	格式化分区
4. 创建挂载点	<code>mkdir /data</code>	创建目录
5. 临时挂载	<code>mount /dev/sdb1 /data</code>	立即可用
6. 永久挂载	编辑 <code>/etc/fstab</code>	重启后自动挂载

第二部分：实践操作

2.1 实验环境准备

实验前提

本实验需要一块未使用的磁盘（可以是虚拟机中添加的虚拟磁盘）。

虚拟机添加磁盘步骤（以 VMware/VirtualBox 为例）：

1. 关闭虚拟机
2. 编辑虚拟机设置
3. 添加硬件 → 硬盘 → 创建新虚拟磁盘
4. 设置磁盘大小（建议 10–20GB）
5. 启动虚拟机

确认新磁盘

```
# 查看系统识别到的所有块设备
lsblk

# 查看所有磁盘的分区信息
fdisk -l

# 只查看磁盘设备
ls -l /dev/sd* /dev/nvme* 2>/dev/null
```

预期输出示例：

```
[root@localhost ~]# lsblk
NAME          MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda            8:0    0   50G  0 disk
├─sda1         8:1    0    1G  0 part /boot
└─sda2         8:2    0   49G  0 part
   ├─rl-root   253:0    0   47G  0 lvm  /
   └─rl-swap   253:1    0    2G  0 lvm  [SWAP]
sdb            8:16    0   20G  0 disk
```

← 这是新添加的磁盘

2.2 实验 1：使用 fdisk 进行 MBR 分区

实验目标

掌握使用 fdisk 创建 MBR 分区的方法。

实验步骤

步骤 1：启动 fdisk

```
# 对新磁盘进行分区
fdisk /dev/sdb
```

fdisk 交互界面：

```
Welcome to fdisk (util-linux 2.32.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0x12345678.

Command (m for help):
```

步骤 2：查看帮助和磁盘信息

```
# 在 fdisk 交互界面中输入：
Command (m for help): m      # 查看帮助

# 常用命令：
# n - 新建分区
# d - 删除分区
# p - 打印分区表
# t - 修改分区类型
# w - 保存并退出
# q - 不保存退出

Command (m for help): p      # 查看当前分区表
```

步骤 3：创建主分区

```
Command (m for help): n      # 新建分区

Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p        # 选择主分区

Partition number (1-4, default 1): 1      # 分区号

First sector (2048-41943039, default 2048): 回车      # 使用默认起始扇区

Last sector, +sectors or +size{K,M,G,T,P} (2048-41943039, default
41943039): +5G
# 创建 5GB 的分区

Created a new partition 1 of type 'Linux' and of size 5 GiB.
```

步骤 4：创建更多分区


```
# 继续创建第二个主分区 (3GB)
Command (m for help): n
Select (default p): p
Partition number (2-4, default 2): 2
First sector: 回车
Last sector: +3G

# 创建第三个主分区 (2GB, 用于 swap)
Command (m for help): n
Select (default p): p
Partition number (3-4, default 3): 3
First sector: 回车
Last sector: +2G
```

步骤 5：修改分区类型为 swap

```
Command (m for help): t      # 修改分区类型
Partition number (1-3, default 3): 3
Hex code (type L to list all codes): L      # 列出所有类型

# 常用类型:
# 82 = Linux swap
# 83 = Linux (默认)
# 8e = Linux LVM

Hex code: 82
Changed type of partition 'Linux' to 'Linux swap'.
```

步骤 6：查看并保存分区表

```
# 查看分区结果
Command (m for help): p

Disk /dev/sdb: 20 GiB, 21474836480 bytes, 41943040 sectors
...
Device      Boot  Start      End  Sectors  Size Id Type
/dev/sdb1           2048 10487807 10485760    5G 83 Linux
```

```
/dev/sdb2      10487808 16779263 6291456 3G 83 Linux
/dev/sdb3      16779264 20973567 4194304 2G 82 Linux swap
```

```
# 保存分区表并退出
```

```
Command (m for help): w
```

```
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

步骤 7：验证分区创建

```
# 刷新分区表（如果内核未自动识别）
```

```
partprobe /dev/sdb
```

```
# 验证分区
```

```
lsblk /dev/sdb
```

```
# 查看详细信息
```

```
fdisk -l /dev/sdb
```

预期输出：

```
[root@localhost ~]# lsblk /dev/sdb
NAME        MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sdb          8:16   0  20G  0 disk
├─sdb1       8:17   0    5G  0 part
├─sdb2       8:18   0    3G  0 part
└─sdb3       8:19   0    2G  0 part
```

2.3 实验 2：使用 gdisk 进行 GPT 分区

实验目标

掌握使用 gdisk 创建 GPT 分区的方法。



实验步骤

步骤 1：安装 gdisk（如果未安装）

```
yum install -y gdisk
```

步骤 2：如果磁盘已有 MBR 分区，先清除

```
# 查看当前分区表类型
gdisk -l /dev/sdb

# 如果要转换为 GPT，需要先清除现有分区
# ⚠️ 警告：这会删除所有数据！
wipefs -a /dev/sdb
```

步骤 3：使用 gdisk 创建 GPT 分区

```
gdisk /dev/sdb
```

gdisk 交互界面：

```
GPT fdisk (gdisk) version 1.0.3

Partition table scan:
  MBR: not present
  BSD: not present
  APM: not present
  GPT: not present

Creating new GPT entries.

Command (? for help):
```

步骤 4：创建分区

```
# 查看帮助
Command (? for help): ?

# 创建第一个分区 (5GB)
Command (? for help): n
Partition number (1-128, default 1): 1
First sector: 回车
Last sector: +5G
Hex code or GUID: 回车    # 默认为 Linux filesystem (8300)

# 创建第二个分区 (3GB)
Command (? for help): n
Partition number (2-128, default 2): 2
First sector: 回车
Last sector: +3G
Hex code or GUID: 回车

# 创建第三个分区 (2GB, swap)
Command (? for help): n
Partition number (3-128, default 3): 3
First sector: 回车
Last sector: +2G
Hex code or GUID: 8200    # Linux swap

# 查看分区表
Command (? for help): p

# 保存并退出
Command (? for help): w
```

步骤 5：验证 GPT 分区

```
# 刷新分区表
partprobe /dev/sdb

# 验证
gdisk -l /dev/sdb
lsblk /dev/sdb
```

2.4 实验 3：使用 parted 分区（支持 MBR 和 GPT）

实验目标

掌握 parted 命令的使用，它同时支持 MBR 和 GPT。

实验步骤

步骤 1：查看磁盘信息

```
# 查看所有磁盘
parted -l

# 进入交互模式
parted /dev/sdb

(parted) print      # 打印分区表
```

步骤 2：创建 GPT 分区表

```
(parted) mklabel gpt      # 创建 GPT 分区表
# 或
(parted) mklabel msdos    # 创建 MBR 分区表
```

步骤 3：创建分区

```
# 创建第一个分区
(parted) mkpart primary xfs 0% 5GB

# 创建第二个分区
(parted) mkpart primary ext4 5GB 8GB

# 创建第三个分区 (swap)
```

```
(parted) mkpart primary linux-swap 8GB 10GB
```

```
# 查看结果
```

```
(parted) print
```

```
# 退出
```

```
(parted) quit
```

步骤 4：使用非交互模式

```
# 一条命令完成分区（脚本化操作常用）
```

```
parted -s /dev/sdb mklabel gpt
```

```
parted -s /dev/sdb mkpart primary xfs 0% 5GB
```

```
parted -s /dev/sdb mkpart primary ext4 5GB 8GB
```

```
parted -s /dev/sdb mkpart primary linux-swap 8GB 10GB
```

```
parted -s /dev/sdb print
```

2.5 实验 4：创建文件系统

实验目标

掌握使用 mkfs 创建不同类型文件系统的方法。

实验步骤

步骤 1：创建 XFS 文件系统

```
# 创建 XFS 文件系统 (RHEL 默认)
mkfs.xfs /dev/sdb1

# 或使用详细选项
mkfs.xfs -f /dev/sdb1      # -f 强制覆盖
mkfs.xfs -L "DATA" /dev/sdb1  # 设置卷标

# 查看文件系统信息
xfs_info /dev/sdb1
```

输出示例：

```
meta-data=/dev/sdb1          isize=512    agcount=4,
agsize=327680 blks
                         =          sectsz=512   attr=2, projid32bit=1
                         =          crc=1        finobt=1, sparse=1,
rmapbt=0
                         =          reflink=1
data            =          bsize=4096    blocks=1310720,
imaxpct=25
                         =          sunit=0     swidth=0 blks
naming          =version 2     bsize=4096   ascii-ci=0, ftype=1
log            =internal log   bsize=4096   blocks=2560, version=2
                         =          sectsz=512   sunit=0 blks, lazy-
count=1
realtime =none              extsz=4096    blocks=0, rtextents=0
```

步骤 2：创建 EXT4 文件系统

```
# 创建 EXT4 文件系统
mkfs.ext4 /dev/sdb2

# 带选项创建
mkfs.ext4 -L "BACKUP" /dev/sdb2  # 设置卷标
mkfs.ext4 -m 1 /dev/sdb2         # 保留空间设为 1% (默认 5%)

# 查看文件系统信息
tune2fs -l /dev/sdb2
```

步骤 3：创建 Swap 分区

```
# 创建 swap 签名
mkswap /dev/sdb3

# 设置标签
mkswap -L "SWAP" /dev/sdb3

# 查看 swap 信息
blkid /dev/sdb3
```

步骤 4：查看文件系统信息

```
# 使用 blkid 查看所有设备的文件系统和 UUID
blkid

# 只查看特定设备
blkid /dev/sdb1

# 使用 lsblk 查看文件系统类型
lsblk -f /dev/sdb
```

输出示例：

```
[root@localhost ~]# lsblk -f /dev/sdb
NAME      FSTYPE LABEL  UUID                                  MOUNTPOINT
sdb
├─sdb1    xfs     DATA   a1b2c3d4-e5f6-7890-abcd-ef1234567890
├─sdb2    ext4    BACKUP  12345678-90ab-cdef-1234-567890abcdef
└─sdb3    swap    SWAP    abcdef12-3456-7890-abcd-ef1234567890
```

2.6 实验 5：挂载和卸载

实验目标

掌握文件系统的挂载和卸载操作。



实验步骤

步骤 1：创建挂载点

```
# 创建挂载目录
mkdir -p /data
mkdir -p /backup

# 验证
ls -ld /data /backup
```

步骤 2：临时挂载

```
# 挂载 XFS 分区
mount /dev/sdb1 /data

# 挂载 EXT4 分区
mount /dev/sdb2 /backup

# 验证挂载
df -h | grep sdb
mount | grep sdb

# 查看挂载详情
findmnt /data
```

输出示例：

```
[root@localhost ~]# df -h | grep sdb
/dev/sdb1      5.0G   68M   5.0G    2% /data
/dev/sdb2      2.9G   9.0M   2.8G    1% /backup
```

步骤 3：启用 Swap

```
# 启用 swap 分区
swapon /dev/sdb3

# 查看 swap 状态
swapon --show
free -h
cat /proc/swaps
```

步骤 4：使用挂载选项

```
# 卸载后重新挂载（带选项）
umount /data

# 只读挂载
mount -o ro /dev/sdb1 /data
touch /data/test      # 会失败，因为只读

# 重新挂载为读写
mount -o remount,rw /data

# 挂载时禁用执行权限
mount -o noexec /dev/sdb2 /backup

# 挂载时不更新访问时间（提升性能）
mount -o noatime /dev/sdb1 /data
```

步骤 5：卸载文件系统

```
# 卸载
umount /data
umount /backup

# 如果提示 "target is busy"
# 方法1：查看谁在使用
lsof /data
fuser -mv /data
```

```
# 方法2：强制卸载（不推荐）
umount -f /data

# 方法3：延迟卸载（等待不再使用时自动卸载）
umount -l /data

# 禁用 swap
swapoff /dev/sdb3
```

2.7 实验 6：配置永久挂载（fstab）

实验目标

掌握 /etc/fstab 配置，实现开机自动挂载。

实验步骤

步骤 1：获取 UUID

```
# 获取分区的 UUID
blkid /dev/sdb1
blkid /dev/sdb2
blkid /dev/sdb3

# 或者一次性查看
blkid /dev/sdb*
```

输出示例：

```
/dev/sdb1: LABEL="DATA" UUID="a1b2c3d4-e5f6-7890-abcd-ef1234567890"
TYPE="xfs"
/dev/sdb2: LABEL="BACKUP" UUID="12345678-90ab-cdef-1234-567890abcdef"
TYPE="ext4"
/dev/sdb3: LABEL="SWAP" UUID="abcdef12-3456-7890-abcd-ef1234567890"
TYPE="swap"
```

步骤 2：备份 fstab

```
# 备份原文件
cp /etc/fstab /etc/fstab.bak

# 验证备份
ls -l /etc/fstab*
```

步骤 3：编辑 fstab

```
# 编辑 fstab
vim /etc/fstab

# 在文件末尾添加以下行：
# 数据分区 - XFS
UUID=a1b2c3d4-e5f6-7890-abcd-ef1234567890 /data xfs defaults
0 0

# 备份分区 - EXT4
UUID=12345678-90ab-cdef-1234-567890abcdef /backup ext4 defaults
0 2

# Swap 分区
UUID=abcdef12-3456-7890-abcd-ef1234567890 swap swap defaults
0 0
```

提示：也可以使用设备名（如 `/dev/sdb1`），但推荐使用 UUID，因为设备名可能在添加/移除磁盘后改变。

步骤 4：验证 fstab 配置

```
# 首先确保所有分区已卸载
umount /data 2>/dev/null
umount /backup 2>/dev/null
swapoff /dev/sdb3 2>/dev/null
```

```
# 使用 mount -a 测试 fstab
mount -a

# 如果有错误，会显示出来
# 如果没有输出，表示成功

# 验证挂载结果
df -h
swapon --show
```

步骤 5：使用 systemd-mount 验证（可选）

```
# 检查 fstab 语法
systemd-analyze verify /etc/fstab

# 查看 systemd 生成的挂载单元
systemctl list-units --type=mount
```

步骤 6：模拟重启验证

```
# 卸载所有手动挂载的分区
umount -a 2>/dev/null

# 重新挂载 fstab 中的所有条目
mount -a

# 验证
df -h
```



fstab 字段详解

字段	说明	常用值
设备	设备标识	UUID=xxx 或 /dev/sdX
挂载点	目录路径	/data, /home, /backup
文件系统	类型	xfs, ext4, swap, nfs
选项	挂载选项	defaults, ro, noexec
dump	备份标记	0（不备份）, 1（备份）
fsck	检查顺序	0（不检查）, 1（根分区）, 2（其他）

常用挂载选项组合

用途	选项
普通数据分区	defaults
优化性能	defaults,noatime
安全性（禁止执行）	defaults,noexec,nosuid
临时目录	defaults,noexec,nosuid,nodev
网络文件系统	defaults,_netdev,nofail
可移动设备	defaults,nofail,x-systemd.automount

2.8 实验 8：磁盘使用分析

实验目标

掌握磁盘空间分析和管理工具。



实验步骤

步骤 1：查看磁盘使用情况

查看所有挂载点使用情况

```
df -h
```

只查看本地文件系统

```
df -hl
```

查看 inode 使用情况

```
df -i
```

查看特定目录

```
df -h /data
```

步骤 2：查看目录大小

查看目录总大小

```
du -sh /data
```

查看子目录大小

```
du -sh /data/*
```

查看目录大小并排序

```
du -sh /var/* | sort -hr | head -10
```

限制深度

```
du -h --max-depth=1 /var
```

显示总计

```
du -sh /var --total
```

步骤 3：查找大文件

```
# 查找大于 100MB 的文件
find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null

# 按大小排序显示前 20 个大文件
find / -type f -exec ls -s {} \; 2>/dev/null | sort -nr | head -20

# 使用 ncdu 交互式查看（需安装）
yum install -y ncdu
ncdu /var
```

步骤 4：使用 lsblk 查看块设备

```
# 树形显示块设备
lsblk

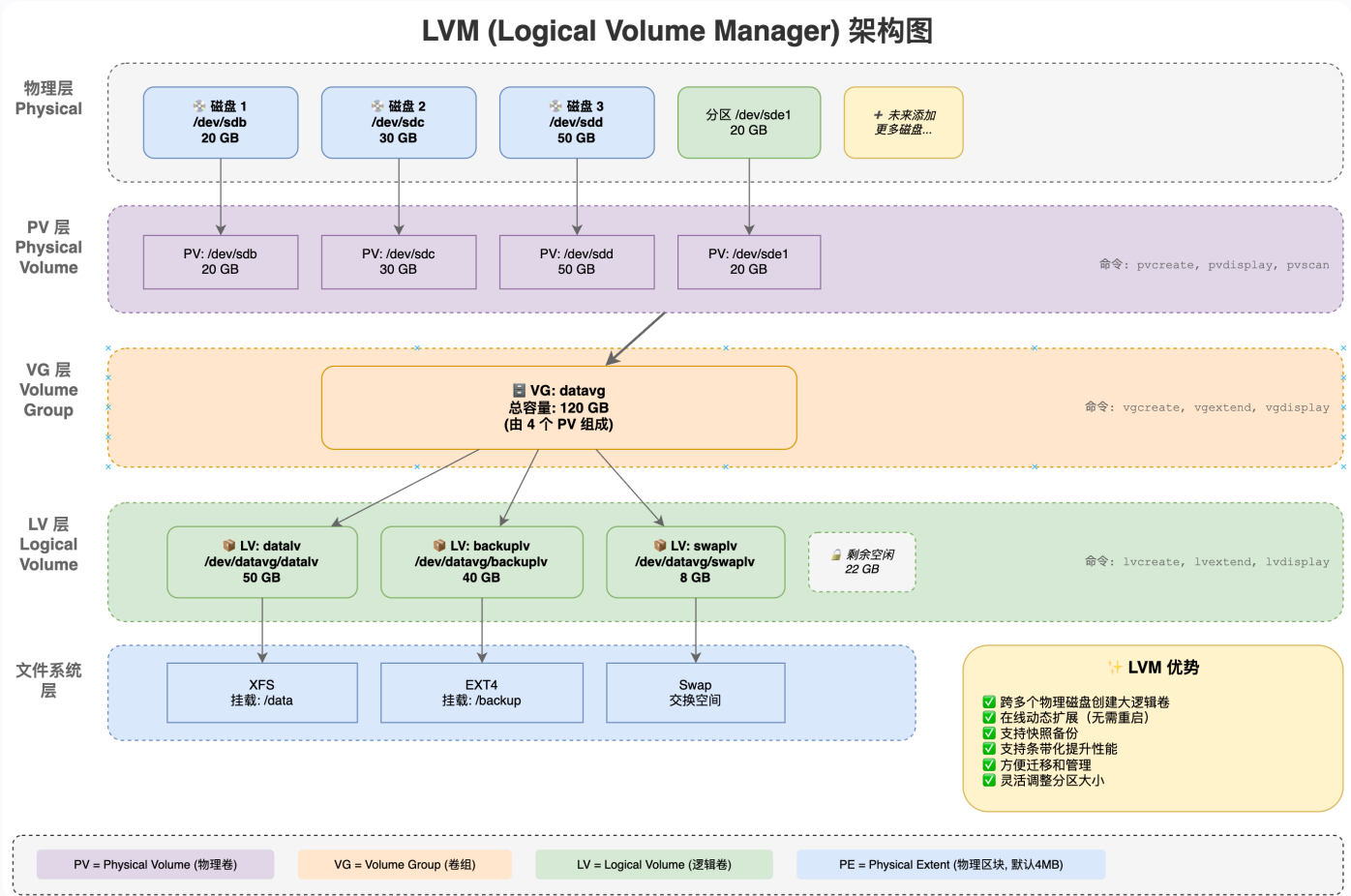
# 显示文件系统和大小
lsblk -f
```

2.9 实验 10：LVM 逻辑卷管理

实验目标

掌握 LVM 的创建、扩容、快照等操作。

配图 5：LVM 架构图



LVM 概念说明

术语	全称	说明
PV	Physical Volume	物理卷，可以是整块磁盘或分区
VG	Volume Group	卷组，由多个 PV 组成的存储池
LV	Logical Volume	逻辑卷，从 VG 中划分出来，相当于分区
PE	Physical Extent	物理区块，VG 的最小分配单位（默认 4MB）

LVM 实验步骤

步骤 1：准备磁盘

```
# 假设已有两块未使用的磁盘 sdb 和 sdc
lsblk
```

```
# 清理磁盘（如果有旧分区）
wipefs -a /dev/sdb
wipefs -a /dev/sdc
```

步骤 2：创建物理卷（PV）

```
# 将磁盘创建为 PV
pvcreate /dev/sdb
pvcreate /dev/sdc
```

```
# 查看 PV 信息
pvs
pvdisplay
```

```
# 详细查看某个 PV
pvdisplay /dev/sdb
```

输出示例：

```
[root@localhost ~]# pvs
PV          VG      Fmt  Attr PSize  PFree
/dev/sdb           lvm2 ---   20.00g  20.00g
/dev/sdc           lvm2 ---   20.00g  20.00g
```

步骤 3：创建卷组（VG）

```
# 创建卷组，包含两个 PV
vgcreate datavg /dev/sdb /dev/sdc
```

```
# 查看 VG 信息
vgs
vgdisplay datavg
```

输出示例：

```
[root@localhost ~]# vgs
VG      #PV #LV #SN Attr   VSize   VFree
datavg   2   0   0 wz--n- 39.99g 39.99g
```

步骤 4：创建逻辑卷（LV）

```
# 创建 10GB 的逻辑卷
lvcreate -L 10G -n datalv datavg

# 使用百分比创建（使用 VG 剩余空间的 50%）
lvcreate -l 50%FREE -n backuplv datavg

# 创建使用所有剩余空间
lvcreate -l 100%FREE -n misclv datavg

# 查看 LV 信息
lvs
lvdisplay
```

输出示例：

```
[root@localhost ~]# lvs
LV      VG      Attr      LSize   Pool Origin Data%  Meta%  Move
Log Cpy%Sync Convert
backuplv datavg -wi-a----- 15.00g

datalv   datavg -wi-a----- 10.00g

misclv   datavg -wi-a----- 14.99g
```

步骤 5：创建文件系统并挂载

```
# 创建文件系统
mkfs.xfs /dev/datavg/data1v
mkfs.ext4 /dev/datavg/backup1v

# 创建挂载点
mkdir -p /data /backup

# 挂载
mount /dev/datavg/data1v /data
mount /dev/datavg/backup1v /backup

# 验证
df -h | grep datavg
```

步骤 6：配置永久挂载

```
# 获取 UUID
blkid | grep datavg

# 添加到 fstab
cat >> /etc/fstab << 'EOF'
/dev/datavg/data1v      /data      xfs        defaults    0 0
/dev/datavg/backup1v    /backup    ext4       defaults    0 2
EOF

# 验证
mount -a
```

步骤 7：扩展逻辑卷（在线扩容）

```
# 查看 VG 剩余空间
vgs

# 扩展 LV（增加 5GB）
lvextend -L +5G /dev/datavg/data1v

# 或扩展到指定大小
```

```
lvextend -L 20G /dev/datavg/data1v

# 或使用所有剩余空间
lvextend -l +100%FREE /dev/datavg/data1v

# 扩展文件系统
# XFS 文件系统
xfs_growfs /data

# EXT4 文件系统
resize2fs /dev/datavg/backup1v

# 验证
df -h /data /backup
```

步骤 8：添加新磁盘扩展 VG

```
# 假设添加了新磁盘 sdd
pvcreate /dev/sdd

# 将新 PV 加入 VG
vgextend datavg /dev/sdd

# 查看扩展后的 VG
vgs

# 现在可以扩展 LV 了
lvextend -l +100%FREE /dev/datavg/data1v
xfs_growfs /data
```

步骤 9：创建 LVM 快照

```
# 创建快照（需要 VG 有足够空闲空间）
lvcreate -L 2G -s -n data1v_snap /dev/datavg/data1v

# 查看快照
lvs
```

```
# 挂载快照（只读）
mkdir -p /mnt/snap
mount -o ro /dev/datavg/data1v_snap /mnt/snap

# 从快照恢复数据
cp /mnt/snap/important_file /data/

# 卸载并删除快照
umount /mnt/snap
lvremove /dev/datavg/data1v_snap
```

LVM 命令速查表

操作	PV	VG	LV
创建	<code>pvcreate</code>	<code>vgcreate</code>	<code>lvcreate</code>
查看	<code>pvs</code> , <code>pvdisk</code>	<code>vgs</code> , <code>vgdisplay</code>	<code>lvs</code> , <code>lvdisplay</code>
扩展	—	<code>vgextend</code>	<code>lvextend</code>
缩小	—	<code>vgreduce</code>	<code>lvreduce</code>
删除	<code>pvremove</code>	<code>vgremove</code>	<code>lvremove</code>
扫描	<code>pvscan</code>	<code>vgscan</code>	<code>lvscan</code>

第三部分：课程总结

命令速查表

磁盘信息查看

命令	说明
<code>lsblk</code>	列出块设备（树形）
<code>lsblk -f</code>	显示文件系统信息
<code>fdisk -l</code>	列出所有磁盘分区
<code>parted -l</code>	列出所有磁盘详情
<code>blkid</code>	显示设备 UUID
<code>df -h</code>	显示磁盘使用情况
<code>df -i</code>	显示 inode 使用情况
<code>du -sh <目录></code>	查看目录大小

分区管理

操作	MBR (fdisk)	GPT (gdisk)	通用 (parted)
进入交互	<code>fdisk /dev/sdb</code>	<code>gdisk /dev/sdb</code>	<code>parted /dev/sdb</code>
新建分区	<code>n</code>	<code>n</code>	<code>mkpart</code>
删除分区	<code>d</code>	<code>d</code>	<code>rm</code>
打印分区	<code>p</code>	<code>p</code>	<code>print</code>
保存退出	<code>w</code>	<code>w</code>	<code>quit</code>

文件系统管理

操作	命令
创建 XFS	<code>mkfs.xfs /dev/sdb1</code>
创建 EXT4	<code>mkfs.ext4 /dev/sdb1</code>
创建 Swap	<code>mkswap /dev/sdb1</code>
挂载	<code>mount /dev/sdb1 /data</code>
卸载	<code>umount /data</code>
启用 Swap	<code>swapon /dev/sdb1</code>
禁用 Swap	<code>swapoff /dev/sdb1</code>
扩展 XFS	<code>xfs_growfs /data</code>
扩展 EXT4	<code>resize2fs /dev/sdb1</code>

LVM 管理

层级	创建	查看	扩展	删除
PV	<code>pvcreate</code>	<code>pvs</code>	—	<code>pvremove</code>
VG	<code>vgcreate</code>	<code>vgs</code>	<code>vgextend</code>	<code>vgremove</code>
LV	<code>lvcreate</code>	<code>lvs</code>	<code>lvextend</code>	<code>lvremove</code>

常见问题解答

Q1：分区后系统没有识别新分区怎么办？


```
# 刷新分区表
partprobe /dev/sdb

# 或重新扫描
echo 1 > /sys/block/sdb/device/rescan
```

Q2: umount 提示 "target is busy" 怎么办?

```
# 查看谁在使用
lsof /data
fuser -mv /data

# 结束进程后再卸载
fuser -km /data
umount /data

# 或延迟卸载
umount -l /data
```

Q3: fstab 配置错误导致系统无法启动怎么办?

1. 重启时进入单用户模式或救援模式
2. 编辑 `/etc/fstab` 修复错误
3. 重启系统

Q4: XFS 文件系统可以缩小吗?

不可以。XFS 只支持在线扩展，不支持缩小。如需缩小，需要备份数据、删除文件系统、重新创建较小的分区、恢复数据。

Q5: 什么时候用 MBR，什么时候用 GPT?

情况	推荐
磁盘 > 2TB	GPT
需要 > 4 个分区	GPT
UEFI 启动	GPT
老旧 BIOS 系统	MBR
新系统默认	GPT

课后练习

练习 1：基础分区操作

1. 使用 fdisk 创建 3 个 MBR 分区
2. 分别格式化为 xfs、ext4、swap
3. 配置永久挂载

练习 2：GPT 分区

1. 使用 gdisk 创建 GPT 分区表
2. 创建 5 个分区
3. 挂载并验证

练习 3：LVM 综合练习

1. 创建包含 2 个 PV 的 VG
2. 创建 2 个 LV
3. 格式化并挂载
4. 添加新磁盘扩展 VG
5. 扩展 LV 和文件系统

6. 创建并恢复快照